AURA - Stage 6: Baselines and External-Contract Integration

Technical implementation and evidence report | October 24, 2026 | v1.0

Executive finding

Stage 6 successfully adds state-only, trace-only, and dual-surface verification on six actual local SQLite/filesystem sandbox executions and implements a conservative importer for the public Agent-Diff benchmark contract format. The combined verifier obtains 5 decisive results out of 6 (all correct in this controlled harness), compared with 3/6 for state-only and 1/6 for trace-only. The remaining run is correctly UNCERTAIN. The full inherited and new test suite passes 171 tests.

However, no official Agent-Diff or AppWorld agent trajectory was executed. Five published contract assertion excerpts were parsed and assessed against 20 hypothetical, locally authored state-diff examples. These are interoperability checks, not independent benchmark outcomes. External runtime integration is an outstanding Stage 6 research dependency.

1. What changed in the codebase

• aura_baselines/surface.py: information-surface ablations (state-only, trace-only, dual).

• aura_external/agentdiff.py: safe subset importer for the upstream answer JSON assertion schema, where conditions, expected change targets, cardinality, unaccounted mutation checks, and fail-closed unsupported cases.

• aura_external/fixtures.py: explicitly synthetic diff generator, never confused with upstream benchmark agent output.

• aura_external/appworld.py: AppWorld readiness check. AppWorld is not installed here.

• reproduce_stage6.py: regenerates 1,200 authored finite-world strategy evaluations, 30 matched planner/local evaluations, 18 evidence-surface ablations, and 20 contract-format checks.

The Stage 1-5 verifier, reasoner, snapshot capture and CERA planner remain part of the package. No remote services or paid model calls are required.

2. Evidence-surface ablation: six real controlled local workflows

The same preexisting task contract and the same quiescent before/after capture were used in each comparison. We intentionally withhold selected probe types for each ablation. This is not an information-parity experiment; it measures the contribution of having state and trace evidence.

Scenario	Controlled expectation	State-only	Trace-only	Dual-surface
clean	PASS	UNCERTAIN	UNCERTAIN	PASS
missing report	FAIL	FAIL	UNCERTAIN	FAIL
collateral	FAIL	FAIL	UNCERTAIN	FAIL
silent noop	FAIL	FAIL	UNCERTAIN	FAIL
denied readback	UNCERTAIN	UNCERTAIN	UNCERTAIN	UNCERTAIN
missing validation	FAIL	UNCERTAIN	FAIL	FAIL

Interpretation: The state-only method finds persistent failures (missing report, unintended row edit, dropped write) but lacks proof of the required validation step. The trace-only method detects the skipped validation but cannot certify persisted outcomes. Dual-surface inspection handles both. The denied-readback case stays UNCERTAIN even with the combined view.

Mode	Local runs	Decisive verdicts	Expected-label matches	False PASS	False FAIL	Mean measured total latency (ms)*
state_only	6	3	4	0	0	2.893
trace_only	6	1	2	0	0	2.785
dual_surface	6	5	6	0	0	3.086

*Total measured latency includes the shared snapshot-capture cost plus performed probes; it excludes tool execution itself. Timing differences in a six-case sandbox should not be generalized to other settings or asserted as statistically meaningful.

3. Information-matched policy comparison (retained Stage 5 baseline)

The planner policies below each receive identical authorized probes, frozen finite model, contract, cost definitions, and access assumptions. This comparison isolates evidence-selection strategy more fairly than the intentionally restricted surface ablation.

Policy	Authored world episodes	Mean modeled marginal cost	Decision coverage	False PASS	False FAIL
fixed	240	3.710067	100%	0	0
greedy	240	3.791661	100%	0	0
cera	240	3.630406	100%	0	0
optimal	240	3.597073	100%	0	0
exhaustive	240	30.273467	100%	0	0

These are 240 enumerated world assignments across closely related templates, evaluated under five policies (1,200 policy executions). They are not 240 independent external tasks. Modeled marginal costs exclude the shared initial snapshot; real capture and probe timings are separately recorded for the six local workflows.

Negative result retained: CERA's improvement over fixed inspection was small, and under cleaner-world priors it tied greedy fact coverage. Current data do not justify a state-of-the-art or statistically significant performance claim.

4. External benchmark interoperability: what was actually verified

The public Agent-Diff GitHub repository documents 224 enterprise API tasks with declarative state-diff assertions. Its checked-in datasets/agent-diff-bench/test.jsonl stores answer as a JSON-encoded string containing assertions and optional ignore_fields. The Stage 6 parser supports changed/added/deleted assertions and a narrow documented set of comparison operators, while recording unsupported clauses instead of silently accepting them.

Source: https://github.com/agent-diff-bench/agent-diff/tree/main/datasets/agent-diff-bench

Five abbreviated excerpts were drawn from public task records box_145, box_125, box_146, box_119, and box_126. They contain 8 illustrative assertion clauses in total; they are not complete upstream test records. For each excerpt, the adapter inspected four locally authored diff situations:

Authored diff situation	Number of checks	Local subset verdict
authored complete	5	PASS

Authored diff situation	Number of checks	Local subset verdict
authored missing required	5	FAIL
authored partial scope	5	UNCERTAIN
authored unlicensed change	5	FAIL

Boundary: These checks establish that our importer and documented local assertion subset behave on simple known fixtures; they do not measure accuracy relative to Agent-Diff's native oracle. Full environment initialization, real agent trajectories, native state snapshots and official scoring were not executed. AppWorld is likewise not installed or evaluated in this runtime.

The optional --agentdiff-jsonl CLI permits format analysis on an externally downloaded full public dataset. That import by itself still does not constitute an agent evaluation.

5. Important safety and validity properties

1. Unsupported external DSL clauses -> UNCERTAIN, never automatic PASS.

2. Missing required changes with incomplete diff coverage -> UNCERTAIN (absence cannot be proved by partial readback).

3. Observed unlicensed mutations or changed fields -> FAIL, even if all required targets were also updated correctly.

4. Duplicate changes without explicit expected count -> UNCERTAIN rather than incorrectly treating an existential match as conclusive.

5. A PASS is conditioned on complete declared mutation-scope evidence; it cannot be inferred from status codes alone.

6. Surface ablations remain explicitly labeled not information-matched; comparing them against CERA is not an algorithmic superiority experiment.

7. External result fixtures are tagged authored, while the contract excerpts retain upstream source identifiers.

6. Reproduction and testing

Run in a fresh extracted AURA_Stage6 directory:

```
python -m pip install -e '.[test]'
python -m pytest -q
python reproduce_stage6.py --out ./new_stage6_results
```

The test suite returned 171 passed locally. The full command recomputed the Stage 5 simulated policy results and captured six new controlled SQLite/filesystem runs; exact wall-clock times will vary. The result directory includes CSV tables, certificates, captured evidence, and run_manifest.json with explicit disclaimers.

7. Threats to validity and remaining work

• Small local sample: Six controlled tasks establish software wiring, not statistical generalization.

• Non-independent synthetic worlds: Repeated assignments to shared Boolean templates do not support real-world confidence intervals on benchmark performance.

• Public contracts are not public rollouts: Parsing external assertions without genuine before/after states and agent traces is not actual independent benchmark evaluation.

• Subset semantics: The local importer approximates a documented subset of Agent-Diff assertions; it is not a faithful, feature-complete replacement for its native scorer. ignore_fields, regex matching, and heterogeneous services need further verification.

• Unobserved external effects: Current complete-state guarantees are scoped to the controlled local SQLite and filesystem capture.

• Capture cost: All policy comparisons must account for mandatory snapshots, not only selective readback costs.

Next scientific gate: Obtain permitted independent agent trajectories or run the actual Agent-Diff/AppWorld sandbox and compare AURA against their native task verifiers on held-out execution outcomes. Until this is done, the external-benchmark part of Stage 6 remains incomplete and Stage 7 can only begin as a controlled pilot, not as a claim of third-party validation.

8. Sources and attribution

• Agent-Diff repository and public task contracts: https://github.com/agent-diff-bench/agent-diff

• Agent-Diff research paper: https://arxiv.org/abs/2602.11224

• AppWorld repository and evaluation instructions: https://github.com/StonyBrookNLP/appworld

• AppWorld research paper: https://arxiv.org/abs/2407.18901

• Internal basis: AURA Stages 1-5 specifications, controlled test harnesses, and saved result files.

GitHub status: No commits or pushes were made to shanmukhdatta/AURA in Stage 6. The repository can be updated once the team decides what evidence and source code to publish.